A Booth-Based Digital Compute-in-Memory Macro With Bitwise-Efficient Multiphase Accumulation

Yiying Jiang, Xiaoqin Wang, Heng You, Qiang Li, and Shushan Qiao

***Abstract*—This brief presents a Booth-based all-digital SRAM compute-in-memory (CIM) macro designed for high-efficiency multiply-and-accumulate (MAC) operations in artificial intelligence applications. The architecture incorporates three key innovations: 1) a pre-encoding (PENC) scheme utilizing a Booth-based consolidated truth table, designed to reduce encoder complexity and computing unit overhead; 2) a minimalist bitwise processing technique, employing only two transistors for bit retention and shifting, aimed to minimize computational complexity within the subcomputing array; and 3) a multiphase product summation approach, integrating unified negative compensation with global sign bit expansion, intended to lower power consumption during partial product accumulation and final addition. The proposed architecture is readily adaptable to multiplications across diverse bit widths and accommodates flexible CIM array sizes, ensuring high scalability. Measurement results from a 55-nm-CMOS process demonstrate that the proposed CIM macro achieves an energy efficiency of 16.8 TOPS/W under 8b/8b MAC operations.**

***Index Terms*—Adder tree, Booth algorithm, compute-in-memory (CIM), energy efficiency, SRAM.**

I. Introduction

Recent advances in artificial intelligence have driven the application of machine learning (ML) across various scientific research fields such as image classification [1], speech recognition [2], natural language processing [3], and so on. The increasing complexity of algorithms and parameters challenges the energy efficiency and throughput of computing platforms. Though application-specific integrated circuit (ASIC) accelerators excel in computation-intensive tasks, their benefits in memory access and data movement remain limited [4]. The rise of data-centric applications has highlighted the "memory wall" problem, where high latency and energy consumption in data transfer become a critical bottleneck for von Neumann architectures [5].

SRAM-based compute-in-memory (CIM), as a typical non-Von Neumann computing architecture that capitalizes on SRAM's maturity and reliability, addresses these limitations by performing computations directly within memory, thereby offering significant performance improvements for data-intensive applications such as neural networks [6], [7]. Existing approaches are primarily digital and analog. Analog CIM faces limitations due to its susceptibility to noise and the performance constraints of analog-to-digital converters (ADCs). In contrast, digital CIM [8] offers higher accuracy by utilizing Boolean logic compared to the analog counterparts.

Received 11 January 2026; revised 29 March 2026; accepted 17 April 2026. Date of publication 4 May 2026; date of current version 11 June 2026. This work was supported by the Smart Grid-National Science and Technology Major Project of China under Grant 2025ZD0806400. *(Corresponding authors: Xiaoqin Wang; Shushan Qiao.)*

Yiying Jiang, Xiaoqin Wang, and Shushan Qiao are with the Institute of Microelectronics of the Chinese Academy of Sciences, Beijing 100029, China, and also with the School of Integrated Circuits, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: wangxiaoqin@ime.ac.cn; qiaoshushan@ime.ac.cn).

Heng You is with Nanjing Institute of Intelligent Technology, Nanjing 211135, China.

Qiang Li is with the Institute of Microelectronics of the Chinese Academy of Sciences, Beijing 100029, China.

Digital Object Identifier 10.1109/TVLSI.2026.3687178

Digital CIM typically employs bit-serial multiplication of the input with data stored in the SRAM array, followed by the accumulation of partial sums through an adder tree to generate the final multiply-and-accumulate (MAC) result [9], [10], [11]. To mitigate the computational overhead associated with multiplication operations, various optimization strategies have been explored. Lin et al. [12] introduce approximate arithmetic to enhance power efficiency, while [13] employs sign-extension-less multiplication circuits to bypass specific sign-bit processing. Tu et al. [14] integrate the Booth algorithm into CIM macro designs as a means to reduce cycle counts and bitwise multiplications. Booth encoding offers efficiency in reducing computational complexity and adaptability to high-performance arithmetic operations, making it a promising candidate for CIM architectures.

Although Booth encoding effectively reduces the number of partial products, challenges remain due to the complexity of internal logic computations and the accumulation circuit. It introduces additional encoding logic and involves complex bit-level operations, such as shifting and inversion, along with the accumulation of signed partial products. These factors result in increased overhead in both computational and configuration circuits, ultimately impacting energy efficiency.

To address the aforementioned issues, in this brief, a novel fully digital CIM macro utilizing the Booth-based PENC algorithm is proposed. The advantages of the proposed macro over existing CIM architectures are summarized as follows.

1) A PENC scheme using a Booth-based consolidated truth table is proposed to reduce input bit width, partial products, and array rows. Unlike prior works targeting general digital data paths [15], [16] or involving complex in-array operations [14], it is co-optimized with computing logic, enabling simplified peripheral circuits and parallel signal generation.
2) Minimal bitwise operations (BOPs), requiring only two transistors, are performed to enable bit retention and shifting, thereby reducing computational complexity within the computing unit.
3) A multiphase accumulation technique, which integrates unified negative value compensation and global sign bit extension, is proposed to notably lower power consumption during the stage of partial sum accumulation and final addition.
4) The proposed CIM macro achieves an energy efficiency of 16.8 TOPS/W under 8b/8b MAC operations.

The remainder of the brief is organized as follows. Section II offers a detailed description of the proposed CIM macro. Section III provides the design implementation, measurement results, and its application in convolutional neural networks. Section IV concludes this brief.

II. Proposed CIM Architecture

A. Macro Overview

The overall architecture of the proposed CIM macro is depicted in Fig. 1, comprising three key components: PENC module, SRAM subcomputing array, and multiphase accumulation. Multiplicands are first encoded by the PENC module employing the Booth-based algorithm, producing control signals that facilitate the generation of partial products. Intermediate results of the partial products are computed

1063-8210 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:41 UTC from IEEE Xplore. Restrictions apply.

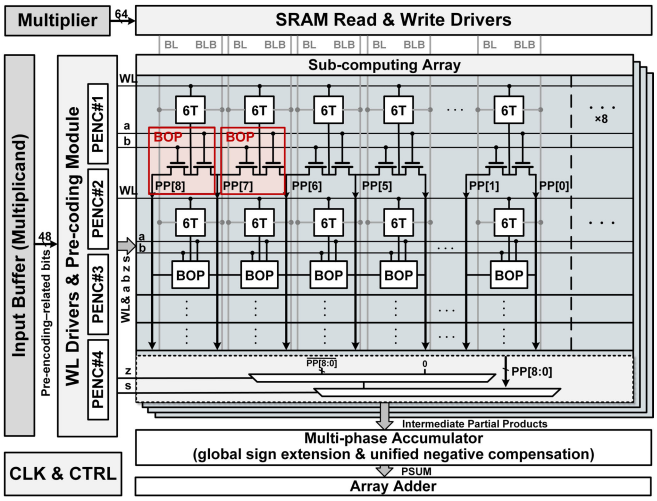


Fig. 1. Proposed CIM macro architecture with internal computing array.

within the SRAM subcomputing array based on the encoding signals, where minimal BOPs, requiring only two transistors, are performed. The multiphase accumulation approach integrating global sign bit extension is then applied to aggregate the partial products, which are subsequently summed by array addition to generate the final MAC output. The following subsections provide a detailed description of the proposed architecture.

B. Booth-Based PENC With Consolidated Truth Table

The PENC scheme is proposed based on a consolidated truth table to reduce the overhead of Booth encoding while simplifying the computational circuit in the computing array, as illustrated in Fig. 2. The scheme is derived solely from the absolute values of partial products (denoted as PP_j for the jth row). The values of the partial products are restricted to the set $\{0, X, 2X\}$, which simplifies the partial product generation process by allowing only bit retention and bit-shifting. Specifically, for $|PP_j| = X$ (indicated by $a = 1$), the partial product remains unchanged. For $|PP_j| = 2X$ (indicated by $b = 1$), the partial product is left-shifted by one bit. Both of these operations are directly executed within the computing array, thereby eliminating the need for external shifting logic. For $|PP_j| = 0$ (indicated by $z = 1$), the partial product is directly set to zero. In addition, in the case when Y is negative (indicated by $s = 1$, for which $s = Y[j + 1]$), the two's complement of the corresponding partial product is calculated. The integrated truth table enables a highly regular design that facilitates parallel signal generation.

The circuit responsible for generating the encoded signals a, b, z, and s is presented in Fig. 2. By grouping three consecutive bits of the multiplicand Y, that is, $Y[j + 1], Y[j], Y[j - 1]$ (j is a positive integer), with one-bit overlap, the number of effective partial products is reduced by half. Every four encoders, organized as a PENC module, are assigned to four rows of the subarray and generate corresponding preencoded control signals. This assignment strategy ensures that the partial products generated within the same cycle across all rows share the same positional weight, allowing direct accumulation and eliminating the need for additional shift or sign-extension logic. Among the generated control signals, a and b are fed into the computational array to guide the generation of intermediate partial products, while z and s are utilized for zero-value detection and sign determination.

C. Computing Array Structure

The computing array is constructed using 6T SRAM bitcells and leverages all-digital bitwise logic to ensure precise computation without loss of accuracy. As demonstrated in Fig. 1, the preencoded signals a and b, are broadcast across each row of the computing

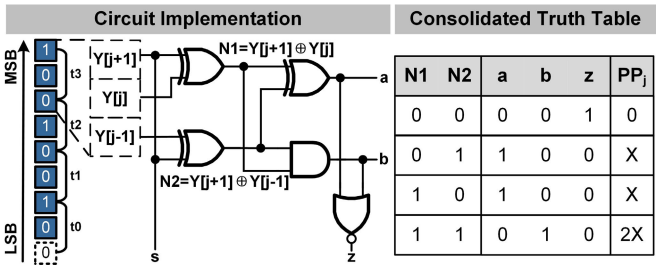


Fig. 2. PENC circuit using consolidated truth table.

array, ensuring load balance and facilitating efficient parallel processing. Benefiting from the proposed PENC scheme, the number of computing units and bitwise multiplications is approximately halved, further enhancing computational efficiency. To support BOPs within the computing array, two transistors are incorporated at the output of each cell in the array to enable bit retention and bit-shifting operations, thereby realizing minimalist bitwise processing. These transistors are directly controlled by the preencoded signals and operate on the stored data without explicit computation. Each bit cell contains two active paths. The bit-retention path, controlled by signal a, which forwards the stored value to the same column, preserving the original value ($\times 1$). The bit-shifting path, controlled by signal b, which forwards the value to the adjacent left column, producing a one-bit left shift corresponding to the $\times 2$ operation. During computation, the preencoded inputs enable one path, allowing each wordline to generate a 9-bit partial product from the 8-bit operand. Each row produces eight partial products in parallel. The resulting partial products are aligned by bit significance along the row direction and accumulated accordingly, eliminating the need for logic-intensive multiplier or inversion circuitry. The control signals z and s are employed to identify zero-valued partial products and determine their sign, respectively, where $z = 1$ signifies a zero result and $s = 1$ denotes a negative partial product, thereby ensuring accurate generation of signed partial products. Within each row of the subarray, these mechanisms generate eight sets of partial product results, which represent the partial product generation process $\{|X|, 2|X|\}$. This approach ensures efficient computation while minimizing the overhead associated with sign handling during the accumulation process.

D. Multiphase Product Accumulation

The proposed PENC scheme reduces the number of partial products for summation, almost halving the cycle count. However, the conversion from one's complement to two's complement, as well as the two-bit left shifts required during the partial product accumulation stage, is unavoidable. These operations introduce additional arithmetic steps and lead to multiple sign-extension processes due to bit-width variations. To address these issues, a multiphase product accumulation method, incorporating unified negative compensation and global sign extension, is proposed to mitigate the overhead associated with partial product accumulation.

The proposed multiphase product accumulation strategy is detailed in Fig. 3. The 64 columns of subcomputing array are partitioned into eight consecutive groups according to the 8-bit operand width. Each group processes one INT8 multiplication, and the accumulation can be expressed as $PSUM = \sum_{i=0}^{3} \left(\sum_{k=0}^{3} PP_i[k] \right) \cdot 2^{2i}$, where the partial product $PP_i[k]$ from the kth row in the ith cycle contributes with weight 2^{2i}. Stage-I performs a preliminary compression with uniform ones-to-twos complement conversion per encoding group. All partial products generated from a PENC group share the same positional weight. A set of 4:2 compressors and specially adapted 4:2 compressors (with one zero input) are used to compress the partial products and the associated sign indicator signals s from the corresponding PENC module. Combined with the column-oriented accumulation strategy, this step eliminates row-wise sign extension

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:41 UTC from IEEE Xplore. Restrictions apply.

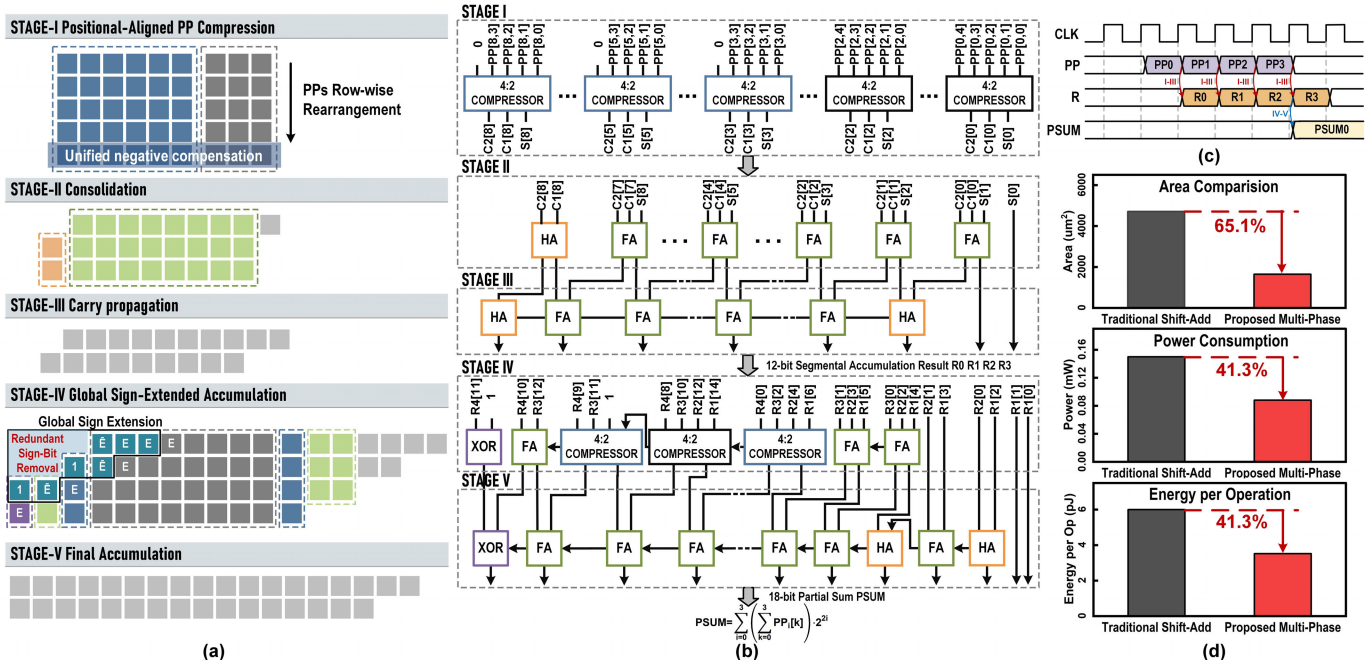


Fig. 3. Illustration of the proposed multiphase accumulation approach, including (a) structure, (b) circuit implementation, (c) signal timing waveform, and (d) comparison with the conventional structure.

and prevents repeated negative compensation. Stage-II reduces the column matrix through a carry-save adder (CSA) tree. The CSA tree consolidates the partial product matrix into two operands, allowing for streamlined addition. Stage-III performs a full carry-propagation addition. The final binary result from this segmental accumulation is computed through a carry propagation adder, yielding a refined value for subsequent computations. The final MAC result is generated through a structured accumulation strategy in Stage-IV and V. To align with the three-bit grouping of the PENC scheme, the total accumulation requires four clock cycles. Specifically, the outputs from the second cycle are left-shifted by two bits relative to the first, with a consistent shift pattern applied to each subsequent cycle to maintain proper arithmetic weighting. Instead of repeatedly extending each partial product, a global injection of fixed "1" is applied to ensure correct sign propagation. This approach facilitates sign bit alignment while reducing redundant 4:2 compressors and ensures consistent bit-growth without local logic overhead. During the accumulation, depending on the characteristics and volume of the processed data, intermediate results in each column are processed using 4:2 compressors, simplified 4:2 compressors, and full adders (FA) with or without fixed inputs, half adders (HA), and XOR gates, as shown in Fig. 3(b).

The circuit implementation is presented in Fig. 3. The multiphase accumulation approach minimizes repetitive multiple sign extensions and reduces unnecessary accumulation operations introduced by two's complement conversions for each partial product. Compared to a traditional Radix-4 Booth shift-add MAC [17] with sequential accumulation and per-step sign extension and correction, the proposed method achieves 41.3% and 65.1% reductions in power and area, respectively, as illustrated in Fig. 3(d).

E. CNN Mapping Strategy

Fig. 4 describes the mapping strategy of the convolution layer onto the proposed CIM macro. The input feature maps ($H \times W \times C$) and filter weights ($K \times K \times C$) are flattened into 1-D vectors. The $K \times K \times C$ kernel weights are serialized and mapped onto the columns of the CIM array. During computation, the input feature values of the current receptive field are processed by the PENC module to generate encoded signals that are then broadcast to the computing array. Parallel MAC operations are performed

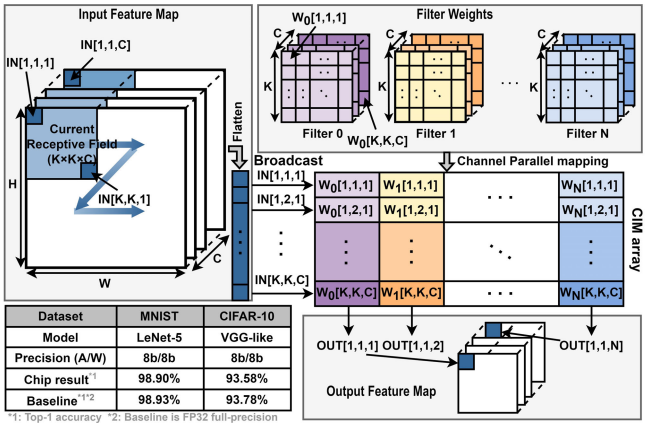


Fig. 4. CNN mapping strategy.

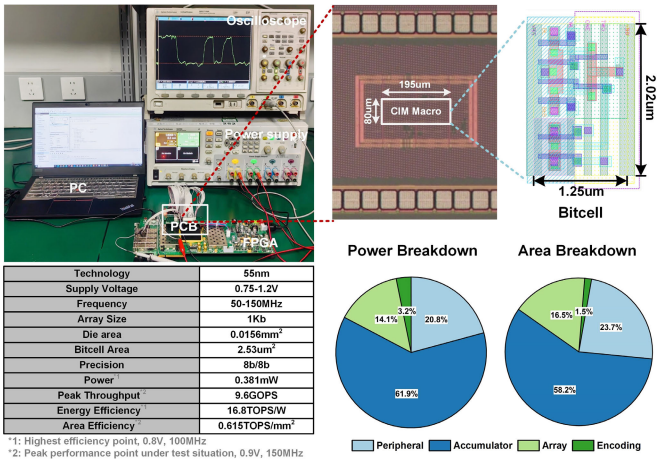
across all columns, with each column accumulating the partial sums corresponding to a distinct filter kernel. Consequently, each column computes the activation of a distinct output channel at a given spatial coordinate, facilitating the parallel generation of N output channels for a single pixel. The sliding-window behavior is realized by updating the broadcast input vector in successive cycles, allowing the convolution filter to traverse the entire input feature map and generate the complete output feature map (OFM). To accommodate convolution filters that exceed the capacity of a single macro, the architecture supports scaling or parallel deployment of multiple instances, sustaining high throughput and utilization. Horizontal tiling increases parallelization across output channels, while vertical tiling spans larger kernels and deeper input-channel dimensions.

III. Design Implementation and Evaluation

A. Design Implementation

The proposed digital SRAM CIM macro was implemented using 55-nm-CMOS technology, and a test chip was fabricated for evaluation. Consistent functionality and performance continue to be

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:41 UTC from IEEE Xplore. Restrictions apply.



Technology	55nm
Supply Voltage	0.75-1.2V
Frequency	50-150MHz
Array Size	1Kb
Die area	0.0156mm^2
Bitcell Area	2.53um^2
Precision	8b/8b
Power *1	0.381mW
Peak Throughput *2	9.6GOPS
Energy Efficiency *1	16.8TOPS/W
Area Efficiency	0.615TOPS/mm^2

*1: Highest efficiency point, 0.8V, 100MHz
*2: Peak performance point under test situation, 0.9V, 150MHz

Fig. 5. Test platform, chip die micrograph, power, and area breakdown.

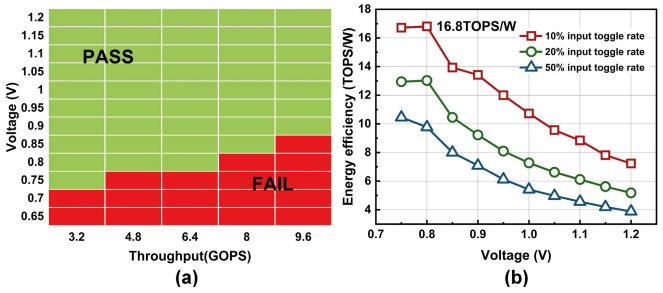


Fig. 6. (a) Shmoo plot of the CIM macro. (b) Energy efficiency under different supply voltage and input toggle rate.

maintained when the array size of the CIM macro is expanded, as both the storage and compute circuits scale proportionally. Fig. 5 presents the die micrograph of the fabricated test chip, followed by the bitcell layout that defines its fundamental unit structure. The power and area breakdown of the CIM macro is then shown to characterize the design's resource utilization and efficiency. To validate the design, the experimental test platform is also illustrated in Fig. 5, enabling comprehensive functional testing and performance evaluation. The prototype operates over a supply voltage range of 0.75–1.2 V, with the test operating frequency between 50 and 150 MHz, considering I/O pad delays, while the CIM macro can function at 50–240 MHz according to postlayout simulations.

B. Performance Analysis and Evaluation

Fig. 6(a) shows the shmoo diagram of the test chip, presenting the MAC operation pass rate and throughput under different voltages. Failures are marked in red, while green marks indicate passing results. The throughput of the CIM macro increases with the supply voltage as higher operational frequency is supported at elevated voltages. Measurement results for the 8b/8b MAC operation show that at a clock frequency of 100 MHz and a supply voltage of 0.8 V, the corresponding throughput is 6.4 GOPS.

Fig. 6(b) presents the measured peak energy efficiency of the CIM macro under different supply voltages and input toggle rates. The energy efficiency of the CIM macro decreases as the input toggle rate increases. This trend arises because the power consumption of digital logic circuits is directly proportional to the toggle rate of the input signals. With the reduction in supply voltage, dynamic power consumption, which serves as the primary contributor to overall power consumption, exhibits a substantial decline, leading to a noticeable improvement in energy efficiency. At a supply voltage of 0.8 V and an input toggle rate of 10%, the CIM macro achieves a

	ISSCC'22[14]	TCASII'23[11]	JSSC'23[18]	JSSC'23[19]	TVLSI'24[13]	JSSC'24[12]	TCASII'25[20]	This Work
Technology	28nm	65nm	28nm	28nm	28nm	28nm	22nm	55nm
MAC operation	Digital	Digital	Analog	Analog	Digital	Digital	Digital	Digital
Frequency (MHz)	50-220	100	N/A	50-80	282-625	250	25-512	50-150
INT Precision	8/16	1-8	4/8	1/4/8/16	8	4(activation) /1(weight)	4	8
Output Precision	-	16b*2	20b	-	23b	11b	13b	21b
Supply Voltage(V)	0.6-1.0	0.8-1.0	0.7-0.9	0.7-0.8	0.65-0.9	0.45-1.1	0.55-1.0	0.75-1.2
Energy Efficiency (TOPS/W)	36.5(8b/8b)	1.2(8b/8b)	15.02-22.75 (8b/8b)	5.98(8b/8b)	21.87(8b/8b)	154(4b/1b)	126(4b/4b)	16.8(8b/8b)
Scaled Energy Efficiency *1	24.09	4.35	22.75*3	4.58	14.44	12.18*4	11.70*4	33.00
Compute Density (TOPS/mm^2)	1.43	N/A	0.468	0.270/0.433	1.111	98	-	0.615*8
Cifar-10 Accuracy	-	-	91.85% – 91.21%*5	91%*6	-	90.41%	-	93.58%*7

*1: Scaled Energy Efficiency = Reported *(Technology/28nm)^(Voltage/0.8V)2
*2: Accumulation processed out of the CIM macro *3: Directly cited the reported peak energy efficiency
*4: Normalized to 8b/8b *5: Resnet-20 *6: Resnet-18 *7: VGG-like
*8: Peak performance point under test situation, 0.9V, 150MHz

Fig. 7. Comparison with existing works.

peak energy efficiency of 16.8 TOPS/W when performing 8b/8b MAC operations at a clock frequency of 100 MHz. The design is validated on classification tasks, achieving 98.90% accuracy on MNIST and 93.58% on CIFAR-10 with negligible accuracy degradation, as shown in Fig. 4.

Fig. 7 presents a comparison with existing works. By employing the proposed Booth-based CIM architecture in conjunction with the proposed multiphase accumulation method, the proposed macro achieves a 1.37×–7.58× improvement in energy efficiency under 8-bit precision, with normalization for process and voltage, compared to existing works.

IV. Conclusion

In this brief, a Booth-based SRAM CIM macro is proposed for efficient MAC operations for AI applications. By incorporating a compact encoding strategy, lightweight bitwise computation, and a structured accumulation mechanism, the design reduces the number of computing units and the overhead of the computational circuit, while enhancing throughput and energy efficiency. Measurement results using 55-nm-CMOS technology demonstrate that the proposed design attains peak energy efficiency of 16.8 TOPS/W for 8b/8b operations with comparable precision, offering a substantial efficiency advantage over existing works. The flexible design can be readily extended to accommodate multiplications of varying bit widths and CIM dimensions, making it suitable for diverse ML algorithms. Incorporating an approximate addition structure into the proposed macro is anticipated to further enhance energy efficiency, with an acceptable accuracy tradeoff for fault-tolerant applications.

References

[1] P. Dhruv and S. Naskar, "Image classification using convolutional neural network (CNN) and recurrent neural network (RNN): A review," in *Machine Learning and Information Processing*. Singapore: Springer, 2020, pp. 367–381.

[2] P. Karmakar, S. W. Teng, and G. Lu, "Thank you for attention: A survey on attention-based artificial neural networks for automatic speech recognition," *Intell. Syst. Appl.*, vol. 23, Sep. 2024, Art. no. 200406. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667305324000802

[3] S. Luo, H. Ivison, S. C. Han, and J. Poon, "Local interpretations for explainable natural language processing: A survey," *ACM Comput. Surv.*, vol. 56, no. 9, pp. 1–36, Apr. 2024, doi: 10.1145/3649450.

[4] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2014, pp. 10–14.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:41 UTC from IEEE Xplore. Restrictions apply.

[5] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, May 2024.

[6] N. Verma et al., "In-memory computing: Advances and prospects," *IEEE Solid State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, Summer. 2019.

[7] S. Mittal, P. Rajput, and S. Subramoney, "A survey of deep learning on CPUs: Opportunities and co-optimizations," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 10, pp. 5095–5115, Oct. 2022.

[8] H. Kim, T. Yoo, T. T.-H. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul. 2021.

[9] Y.-D. Chih et al., "16.4 An 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.

[10] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm^2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[11] X. Zhang, Y. Lu, B. Wang, and T. T.-H. Kim, "A digital bit-reconfigurable versatile compute-in-memory macro for machine learning acceleration," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 5, pp. 1744–1748, May 2023.

[12] C.-T. Lin et al., "DIMCA: An area-efficient digital in-memory computing macro featuring approximate arithmetic hardware in 28 nm," *IEEE J. Solid-State Circuits*, vol. 59, no. 3, pp. 960–971, Mar. 2024.

[13] X. Si et al., "A 28 nm 16-kb sign-extension-less digital-compute-in-memory macro with extension-friendly compute units and accuracy-adjustable adder-tree," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 11, pp. 2164–2168, Nov. 2024.

[14] F. Tu et al., "A 28 nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[15] Q. Wu et al., "MHE-TPE: Multi-operand high-radix encoder for mixed-precision fixed-point tensor processing engines," in *Proc. 58th IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2025, pp. 1625–1639, doi: 10.1145/3725843.3756101.

[16] Q. Cheng et al., "A low-power sparse convolutional neural network accelerator with pre-encoding radix-4 booth multiplier," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 6, pp. 2246–2250, Jun. 2023.

[17] S. Park and D. Park, "Bit-separable radix-4 booth multiplier for power-efficient CNN accelerator," in *Proc. IEEE Symp. Low-Power High-Speed Chips (COOL CHIPS)*, Apr. 2024, pp. 1–6.

[18] J.-W. Su et al., "A 8-b-precision 6T SRAM computing-in-memory macro using segmented-bitline charge-sharing scheme for AI edge chips," *IEEE J. Solid-State Circuits*, vol. 58, no. 3, pp. 877–892, Mar. 2023.

[19] C.-Y. Yao, T.-Y. Wu, H.-C. Liang, Y.-K. Chen, and T.-T. Liu, "A fully bit-flexible computation in memory macro using multi-functional computing bit cell and embedded input sparsity sensing," *IEEE J. Solid-State Circuits*, vol. 58, no. 5, pp. 1487–1495, May 2023.

[20] M. Lou et al., "Area-efficient and low-power 8T compute-SRAM bitcell design for digital compute-in-memory macros in 22 nm CMOS," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 72, no. 11, pp. 1605–1609, Nov. 2025.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:41 UTC from IEEE Xplore. Restrictions apply.